

Assignment 2. Factory Method & Abstract Factory

Design Patterns. Theme: Smart-Home Hub

Student: _____ Group: _____ Date: 15.09.2026

GitHub: <https://github.com/shaxinhoo/assignment2-design-patterns>

1. Introduction

In Assignment 1 the problem was a long constructor. In this assignment the problem is different: the client code writes `new ConcreteClass()` everywhere and grows an if / switch ladder every time a new variant appears. I solve it with two creational patterns: Factory Method for one product and Abstract Factory for a family of products.

My theme is a Smart-Home Hub. In Part A the hub installs one device (a lamp, a thermostat or a camera). In Part B the hub works with a full vendor kit: a sensor, a controller and a mobile app that must all come from the same vendor, Nordic or Solaris.

2. Part A. Factory Method

We have 1 interface, 1 abstract class and 7 classes:

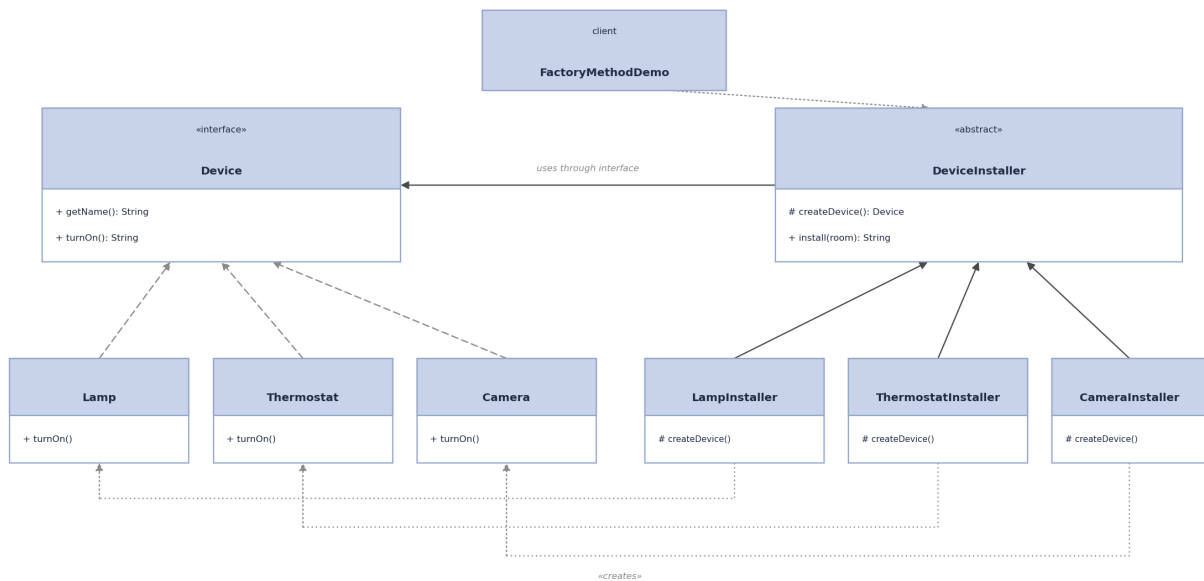
interface `Device` is the product. It has two real methods, `getName()` and `turnOn()`.

class `Lamp`, class `Thermostat` and class `Camera` are the concrete products. Each of them gives its own output when it is turned on.

abstract class `DeviceInstaller` is the creator. It declares the factory method `createDevice()` and has the business method `install(room)`, which calls the factory method and then works with the device only through the `Device` interface.

class `LampInstaller`, class `ThermostatInstaller` and class `CameraInstaller` are the concrete creators. Each one overrides `createDevice()` and decides which device to create.

class `FactoryMethodDemo` is the client. It calls `install()` on an installer and never writes `new Lamp()` itself.



Factory Method: one product, subclasses decide which one

Product

```
public interface Device {
```

```
    String getName();
```

```
    String turnOn();
```

```
}
```

Concrete products

```
public class Lamp implements Device {
```

```
    private static final int DEFAULT_BRIGHTNESS = 70;
```

```
    @Override
```

```
    public String getName() {
```

```
        return "Lamp";
```

```
    }
```

```
    @Override
```

```
    public String turnOn() {
```

```
        return "light is on, brightness " + DEFAULT_BRIGHTNESS + "%";
```

```
    }
```

```
}
```

```
public class Thermostat implements Device {
```

```
    private static final int DEFAULT_TEMPERATURE = 22;
```

```
    @Override
```

```
    public String getName() {
```

```
        return "Thermostat";
```

```
    }
```

```
    @Override
```

```
    public String turnOn() {
```

```
        return "heating to " + DEFAULT_TEMPERATURE + " C";
```

```
    }
```

```

}

public class Camera implements Device {

    private static final String RESOLUTION = "1080p";

    @Override
    public String getName() {
        return "Camera";
    }

    @Override
    public String turnOn() {
        return "recording in " + RESOLUTION;
    }
}

```

Creator with the factory method and the business method

```

public abstract class DeviceInstaller {

    protected abstract Device createDevice();

    public String install(String room) {
        Device device = createDevice();
        return device.getName() + " installed in " + room + ": " + device.turnOn();
    }
}

```

Concrete creators

```

public class LampInstaller extends DeviceInstaller {

    @Override
    protected Device createDevice() {
        return new Lamp();
    }
}

public class ThermostatInstaller extends DeviceInstaller {

    @Override
    protected Device createDevice() {
        return new Thermostat();
    }
}

public class CameraInstaller extends DeviceInstaller {

    @Override
    protected Device createDevice() {
        return new Camera();
    }
}

```

Client

```

public class FactoryMethodDemo {

    private static final String LIVING_ROOM = "living room";
    private static final String BEDROOM = "bedroom";
    private static final String HALLWAY = "hallway";
}

```

```

public static void main(String[] args) {
    System.out.println("=== Part A: Factory Method ===");
    setUp(new LampInstaller(), LIVING_ROOM);
    setUp(new ThermostatInstaller(), BEDROOM);
    setUp(new CameraInstaller(), HALLWAY);
}

private static void setUp(DeviceInstaller installer, String room) {
    System.out.println(installer.install(room));
}
}

```

3. Part B. Abstract Factory

We have 4 interfaces and 9 classes:

interface Sensor, interface Controller and interface MobileApp are the abstract products. They belong together, because a real smart-home kit always has all three.

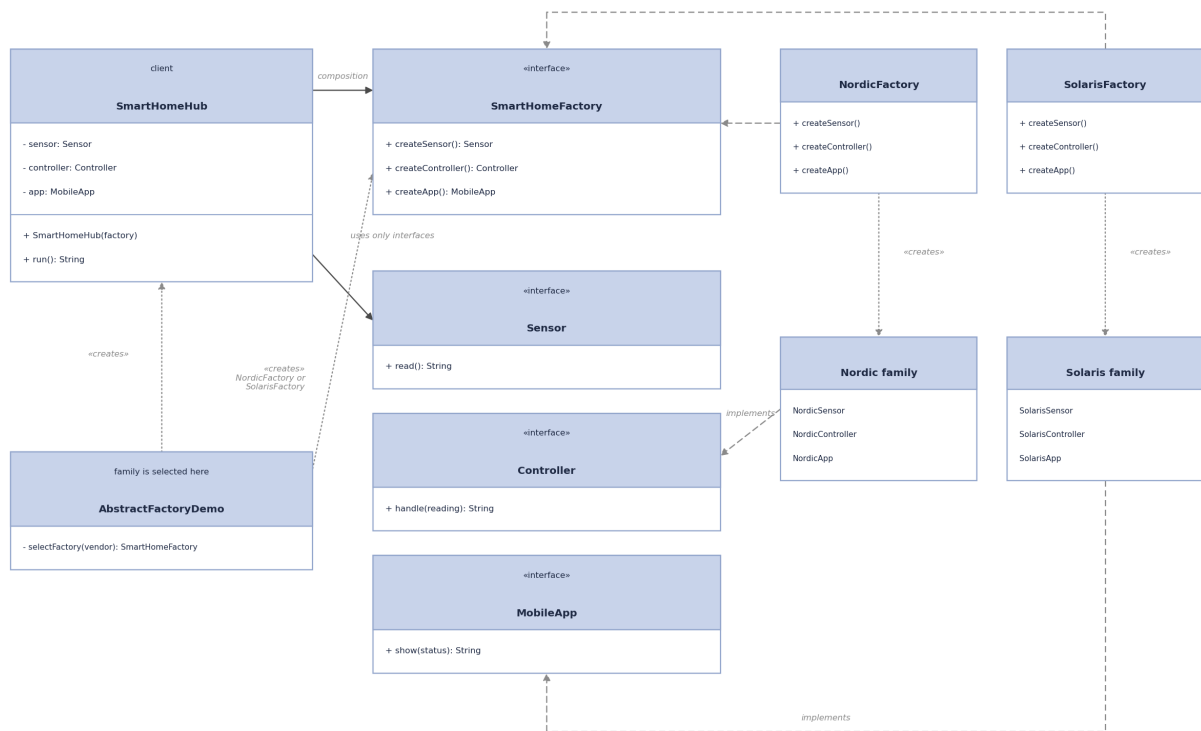
interface SmartHomeFactory is the abstract factory. It has one create-method per product: createSensor(), createController(), createApp().

class NordicFactory and class SolarisFactory are the concrete factories. NordicFactory returns only Nordic products and SolarisFactory only Solaris products, so it is impossible to mix a Nordic sensor with a Solaris app.

class NordicSensor, NordicController, NordicApp and class SolarisSensor, SolarisController, SolarisApp are the two product families.

class SmartHomeHub is the client. It receives the factory through the constructor (composition) and uses only the interfaces. It does not know the concrete classes.

class AbstractFactoryDemo is the only place where the family is selected. To move the whole house to another vendor, one line changes.



Abstract Factory: a family of products, switched all at once

Abstract products

```

public interface Sensor {

    String read();
}

public interface Controller {

    String handle(String reading);
}

public interface MobileApp {

    String show(String status);
}
  
```

Abstract factory

```

public interface SmartHomeFactory {

    Sensor createSensor();

    Controller createController();

    MobileApp createApp();
}
  
```

Concrete factory and its family (Nordic, Solaris is the same)

```

public class NordicFactory implements SmartHomeFactory {

    @Override
    public Sensor createSensor() {
        return new NordicSensor();
    }
  }
  
```

```

    }

    @Override
    public Controller createController() {
        return new NordicController();
    }

    @Override
    public MobileApp createApp() {
        return new NordicApp();
    }
}

public class NordicSensor implements Sensor {

    private static final int TEMPERATURE = 19;

    @Override
    public String read() {
        return "Nordic sensor: " + TEMPERATURE + " C";
    }
}

public class NordicController implements Controller {

    @Override
    public String handle(String reading) {
        return "Nordic controller got [" + reading + "] and turned the heating on";
    }
}

public class NordicApp implements MobileApp {

    @Override
    public String show(String status) {
        return "Nordic app (dark theme): " + status;
    }
}

```

Client that gets the factory by composition

```

public class SmartHomeHub {

    private final Sensor sensor;
    private final Controller controller;
    private final MobileApp app;

    public SmartHomeHub(SmartHomeFactory factory) {
        this.sensor = factory.createSensor();
        this.controller = factory.createController();
        this.app = factory.createApp();
    }

    public String run() {
        String reading = sensor.read();
        String status = controller.handle(reading);
        return app.show(status);
    }
}

```

The one place where the family is selected

```

public class AbstractFactoryDemo {

    private static final String NORDIC = "nordic";
    private static final String SOLARIS = "solaris";

    public static void main(String[] args) {
        System.out.println("=== Part B: Abstract Factory ===");
        String vendor = args.length > 0 ? args[0] : NORDIC;
        SmartHomeFactory factory = selectFactory(vendor);
        SmartHomeHub hub = new SmartHomeHub(factory);
        System.out.println(hub.run());
    }

    private static SmartHomeFactory selectFactory(String vendor) {
        switch (vendor) {
            case NORDIC:
                return new NordicFactory();
            case SOLARIS:
                return new SolarisFactory();
            default:
                throw new IllegalArgumentException("Unknown vendor: " + vendor);
        }
    }
}

```

4. Factory Method vs Abstract Factory

Factory Method creates one product. It works through inheritance: DeviceInstaller declares createDevice() and the subclasses override it. To get a different product you take a different subclass.

Abstract Factory creates a whole family. It works through composition: SmartHomeHub does not extend anything, it receives a SmartHomeFactory object in the constructor and asks it for all three products. To get a different family you pass a different factory object.

Part B is not just three Factory Methods put together. The point is that one factory returns three products that are guaranteed to fit each other, and the client switches all of them at once by changing one object.

SOLID. Open/Closed: to add a new device (for example a Speaker) I add class Speaker and class SpeakerInstaller, and DeviceInstaller and the client do not change. To add a new vendor I add one factory and three product classes, and SmartHomeHub does not change. Single Responsibility: creation and business logic are separated. DeviceInstaller.install() only uses the device, the subclasses only create it. SmartHomeHub only runs the kit, it never creates anything.

5. Program output

=== Part A: Factory Method ===

Lamp installed in living room: light is on, brightness 70%

Thermostat installed in bedroom: heating to 22 C

Camera installed in hallway: recording in 1080p

=== Part B: Abstract Factory ===

Nordic app (dark theme): Nordic controller got [Nordic sensor: 19 C] and turned the heating on

=== Part B: Abstract Factory === (run with argument "solaris")

Solaris app (light theme): Solaris controller got [Solaris sensor: 31 C] and turned the cooling on

6. Conclusion

Both patterns remove new ConcreteClass() from the client and replace the if / switch ladder with polymorphism. Factory Method is enough when there is one product with several variants. Abstract Factory is needed when several products must be consistent with each other.

The drawbacks are also clear. The number of classes grows fast: Part B has 13 files for a very small program. The weak spot of Abstract Factory is a new kind of product: if I add a Camera to the vendor kit, I have to change the SmartHomeFactory interface and every concrete factory. And if a program has only one vendor and one device type, both patterns are over-engineering and a plain constructor is better.

GitHub repository: <https://github.com/shaxinhoo/assignment2-design-patterns>